

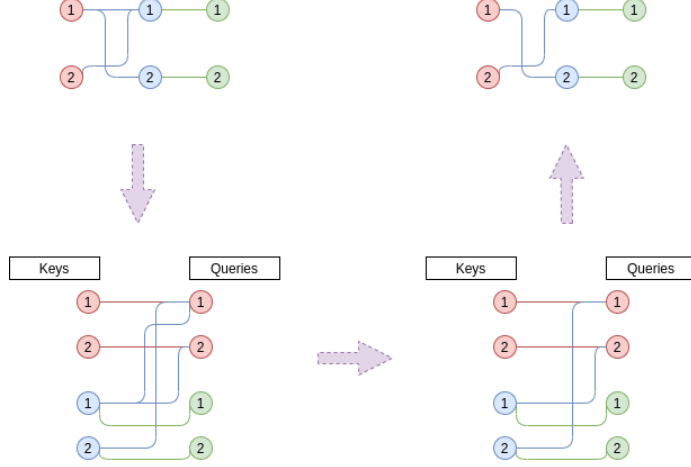


Figure 3: (1, 1): Instance of 3-DM. We denote with red color the X vertex set, with blue the Y vertex set and with green the Z vertex set. (2, 1): Ours transformation for the reduction. Red and blue edges have reward r_1 , while green edges have reward $r_2 \ll r_1$. Missing edges have reward 0. We create one query and one key for each vertex of X . We also create one key (blue color) for each vertex of Y and one query (green color) for each vertex of Z . Connections between red queries and blue keys, as well as, connections between blue keys and green queries follow the problem instance. (2, 2): Optimal queries, keys clustering in groups of 2 for the max-mass 7 problem. (1, 2): Transformation of (2, 2) solution back to the 3-DM instance.

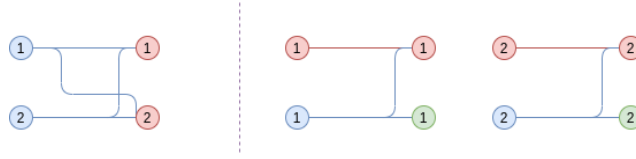


Figure 4: Illustration of potential clusterings. (1, 1): sub-optimal clustering. (1, 2): optimal clusterings. Even though the clustering at the left side obtains more mass compared to any of the clusterings in the right side, it loses entirely the rewards that red keys can give. Indeed, clustering on the left side has one r_1 reward more than any of the two clusterings on the right, but in further clusterings red keys $\{1, 2\}$ will not be matched with anything (by construction) and thus a total reward of $2r_1$ will be lost.

We are given sets $\mathcal{Q}, \mathcal{K}$ and a number L . For each $q_i \in \mathcal{Q}$, we create a key vector k_{q_i} such as $q_j \cdot k_{q_i} = \begin{cases} a, & \text{if } i = j \\ -\infty, & \text{o/w} \end{cases}$, where a is a positive constant the choice of which we will determine later in this proof.

We denote the augmented key set with $\mathcal{K}'$.

We will now solve, with our hypothetical polynomial algorithm, the following optimization problem for our new input set:

$$\min_{\mathcal{C}^L} \|\sigma(\hat{P}_0) - \sigma(P)\|_F^2$$

It holds that:

$$\begin{aligned} \min_{\mathcal{C}^L} \|\sigma(\hat{P}_0) - \sigma(P)\|_F^2 &= \min_{\mathcal{C}_t^L \in \mathcal{C}^L} \sum_{(q,k) \notin \mathcal{C}_t^L} \left(\frac{e^{q \cdot k}}{q_D} \right)^2 + \sum_{(q,k) \in \mathcal{C}_t^L} \left(\frac{e^{q \cdot k}}{q_D} - \frac{e^{q \cdot k}}{q_{C_t^L}} \right)^2 \\ &= \max_{\mathcal{C}_t^L \in \mathcal{C}^L} \sum_{(q,k) \in \mathcal{C}_t^L} e^{2q \cdot k} \cdot \left(\frac{2}{q_D \cdot q_{C_t^L}} - \frac{1}{q_{C_t^L}^2} \right), \end{aligned}$$

where q_D denotes the denominator of the dense softmax and $q_{C_t^L}$ denotes the denominator of the cluster softmax, i.e. $q_D = \sum_{k \in \mathcal{K}} e^{q \cdot k}$ and $q_{C_t^L} = \sum_{k \in C_t^L} e^{q \cdot k}$ for a given cluster $C_t^L \in \mathcal{C}^L$.

We will now show that for a proper choice of a , this problem is equivalent to:

$$\max_{C_t^L \in \mathcal{C}} \sum_{(q,k) \in C_t^L} e^{2q \cdot k}.$$

Let $R_q = \frac{2}{q_D q_{C_t^L}} - \frac{1}{q_{C_t^L}^2}$. As we increase the value of a , the inner product of its query with its' special key gets significantly bigger compared to other inner products and thus for large enough values of a , we know that each query will get clustered with its' special key. We can control how close $q_D, q_{C_t^L}$ are by setting appropriately the a value. Specifically, we choose a such that $q_D(1 - \epsilon) < q_{C_t^L}, \forall q \in \mathcal{Q}, C_t^L \in \mathcal{C}^L$, where $\epsilon = \epsilon(a)$ a small positive constant the choice of which we will determine soon. By definition, $q_{C_t^L}$ is always smaller than q_D , and thus we for that choice of a we have $q_D(1 - \epsilon) < q_{C_t^L} < q_D$. Then, $R_q > \frac{2}{q_D^2} - \frac{1}{q_D^2(1-\epsilon)^2} = \frac{1}{q_D^2}(2 - \frac{1}{(1-\epsilon)^2}) = \frac{1-\epsilon'}{q_D^2}$ where $1 + \epsilon' = \frac{1}{(1-\epsilon)^2}$. But also, $R_q = \frac{2q_{C_t^L} - q_D}{q_{C_t^L}^2 q_D} < \frac{2q_D - q_D}{q_{C_t^L}^2 q_D} = \frac{1}{q_{C_t^L}^2} < \frac{1}{(1-\epsilon)^2 q_D} = (1 + \epsilon') \frac{1}{q_D^2}$. Then, we have that:

$$\frac{1 - \epsilon'}{q_D^2} < R_q < \frac{1 + \epsilon'}{q_D^2}. \quad (8)$$

Now consider the following optimization problems:

$$\begin{cases} P_0 : & \max \sum_{(q,k) \in C_t^L} e^{2q \cdot k} R_q \\ P_1 : & \max \sum_{(q,k) \in C_t^L} \frac{e^{2q \cdot k}}{q_D} \end{cases}.$$

Let $F(c), G(c)$ the objective functions of P_0, P_1 respectively.

Using (8), we get that:

$$(1 - \epsilon')G(c) \leq F(c) \leq (1 + \epsilon')F(c). \quad (9)$$

Our claim is that for a suitable choice of ϵ' , i.e. for a suitable choice of a , it holds that $\operatorname{argmax} P_0 = \operatorname{argmax} P_1$ ⁴. We prove that by contradiction. Let c_1 be the optimal choice of P_0 and c_2 be the optimal choice of P_1 . Then, we know that $F(c_1) > F(c_2)$ and $G(c_2) > G(c_1)$. Using (9), we get that:

$$(1 - \epsilon')G(c_1) - (1 + \epsilon')G(c_2) < F(c_1) - F(c_2) < (1 + \epsilon')G(c_1) - (1 - \epsilon')G(c_2). \quad (10)$$

We denote with d the gap between the optimal value $F(c_1)$ and the non optimal solution $F(c_2)$, i.e. $d = F(c_1) - F(c_2)$. Then, from (10), we get that:

$$d < (1 + \epsilon')G(c_1) - (1 - \epsilon')G(c_2) - (1 - \epsilon')G(c_1) + (1 + \epsilon')G(c_2) = 2\epsilon'(G(c_1) + G(c_2)).$$

Let θ_1 the maximum value of $G(c_1) + G(c_2)$ among all the clusterings $c_1, c_2 \in \mathcal{C}^L$, i.e. among all the possible valid clusterings in L groups. Then, $d < 2\epsilon'\theta_1$. However, since F is a function that maps from discrete clusterings to real numbers, two non-optimal solutions of $F(c)$ differ for at least a minimum distance. In that case, the minimum distance should be at least $e^{p_{\min}} R_{\min}$, where $p_{\min}$ is the minimum product between any query and any key and $R_{\min}$ is the minimum value that R can take for any clustering. Let $\theta_2 = e^{p_{\min}} R_{\min}$. Then, $d \geq \theta_2$. If we choose ϵ' such that: $2\epsilon'\theta_1 < \theta_2$ then we have a contradiction. This is always possible since we can set the value of ϵ' to arbitrarily small values as we grow a arbitrarily big. Thus, we proved that the problems P_0, P_1 have the same argmax for a proper choice of a . Then, for that choice of a the problem $\min_{C^L} \|\sigma(\hat{P}_0) - \sigma(P)\|_F^2$ is equivalent to P_1 which is equivalent to the problem:

$$\max_{C_t^L \in \mathcal{C}} \sum_{(q,k) \in C_t^L} e^{2q \cdot k},$$

since q_D does not affect the choice of optimal clusters.

⁴We assume that if there is a set of optimal solutions, then we pick with the same order from that set for both problems.

In the latter problem, we can replace all queries q and keys k with new vectors q', k' such that: $q' \cdot k' = e^{2q \cdot k}$. This is equally hard to solving:

$$\max_{\mathcal{C}_t^L \in \mathcal{C}} \sum_{(q,k) \in \mathcal{C}_t^L} q \cdot k$$

which we proved to be NP-hard. □

10 Code

To encourage further research in sparse attention models, we open-source all our code and we release a Python package, named `smurf`. The repository for the code is the following: <https://github.com/giannisdaras/smyrf>. `smurf` implements SMYRF attention for Pytorch [36]. We plan to release implementation for Tensorflow [70] soon as well. `smurf` contains various examples on pre-training and finetuning state-of-the-art models for Computer Vision and Natural Language Processing tasks. Regarding examples, at the moment `smurf` includes:

- a TPU-compatible implementation of SMYRF-BigGAN, based on the official Pytorch implementation (<https://github.com/ajbrock/BigGAN-PyTorch>) for GPUs.
- code for training SMYRF-BigGAN on Celeba-HQ on a single TPU device.
- interactive notebooks showing how to use a pre-trained BigGAN for image generation with SMYRF on Celeba-HQ and ImageNet.
- tools to visualize cluster memberships for pixels of SMYRF generated images.
- code for replacing dense attention with SMYRF layers for state-of-the-art pre-trained NLP models, compatible with HuggingFace’s Transformers [71] library.
- interactive notebooks for fine-tuning pre-trained NLP models on GLUE [25] and IMDB [52].
- tools for profiling SMYRF’s performance compared to dense attention.

We also share the weights of SMYRF-BigGAN trained on Celeba-HQ at resolutions 128×128 and at 256×256 with attention at 128×128 , 256×256 respectively. Although these models are outperformed by non-attention GANs (e.g. StyleGAN [72, 73]), we believe that releasing them will help researchers understand better attention at higher resolutions. Hopefully, SMYRF will motivate the usage of more attention layers on new GAN architectures.

11 Singular values decay for pre-trained models

As noted in the paper, row-wise softmax can change the rank of a matrix. For example, the matrix $\begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}$ has rank 1, while the matrix $\sigma\left(\begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}\right) = \begin{bmatrix} 0.7311 & 0.2689 \\ 0.8808 & 0.1192 \end{bmatrix}$ has rank 2. Back to the context of attention, we have defined the product matrix $P = Q \cdot K^T$, where $Q : \mathbb{R}^{|\mathcal{Q}| \times d}$ represents the queries matrix and $K : \mathbb{R}^{|\mathcal{K}| \times d}$ the keys matrix. By the definition of rank, if the embeddings dimension is smaller than the sequence length dimension, i.e. $d < \min(|\mathcal{Q}|, |\mathcal{K}|)$, then P is low rank. However, the attention matrix after softmax, i.e. $\sigma(P)$, could be a full rank matrix. In this section, we provide experimental evidence that attention maps produced by pre-trained models are actually near low-rank.

Figures 5, 6 depict the singular values of the attention maps (for a random input⁵) for a pre-trained BigGAN (attention map dimensions: 4096×1024) and a pre-trained BERT (shown attention map dimensions: 64×64 , 256×256). For the pre-trained BigGAN (Figure 5) the decay in singular values is exponential. Specifically, in Figure 5 most singular values are very close to 0, which means that the attention map is effectively low rank. Figure 6 shows decay of singular values for a pre-trained BERT for sequence lengths: (a) 64, (b) 128. We illustrate decay for 144 heads (12 heads for each one of the 12 layers). For the majority of heads, singular values decay exponentially. We also see that the heads that do not demonstrate exponential decay in the singular values maintain this property

⁵We experimented with different random inputs and there is no qualitative difference in the decay of singular values)

for both inputs (e.g. see the red line in both plots). In our experiments, we find that these heads are harder to approximate with SMYRF.

Singular values of BigGAN's attention map (after softmax) for 6 randomly generated images.

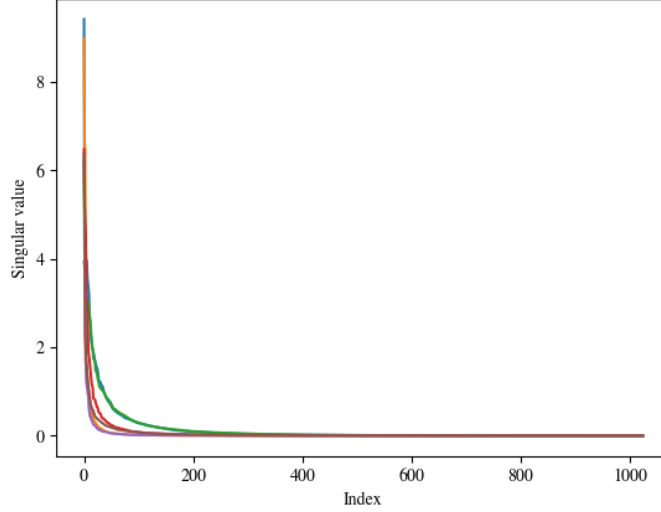


Figure 5: Decay of singular values of the attention map (after softmax) of a pre-trained BigGAN. Decay of singular values is exponential, which means that the matrix after softmax is effectively low rank.

12 Cluster memberships for generated images of a pre-trained BigGAN

In this section, we visualize how SMYRF’s adaptive clustering algorithm assigns queries in clusters for a pre-trained BigGAN. This inspection gives useful insights into how the algorithm actually works in practice.

Top row of Figure 7 shows a random maltese dog generated by a pre-trained BigGAN [1]. The second row, illustrates how a single SMYRF hashing round assigns queries and keys for this particular image in two clusters: the first cluster is denoted with gray and the second with white color. As shown, SMYRF assignments preserve locality while enabling the modeling of arbitrary complex dependencies between input pixels. Indeed, pixels in the same neighborhoods are mostly organized in the same cluster. This observation is even more pronounced for background pixels (see big gray blocks). However, we also see that distant pixels sometime belong to the cluster as well. By only looking at the assignments in clusters (second row), we can infer that the image is roughly separated in three parts: the top part (mostly gray pixels), the middle part (mostly white pixels) and the bottom part (mostly gray pixels). These parts correspond to the top background, the dogs’ face and the bottom background respectively. Third row of Figure 7 illustrates (for the same image) assignments in 128 clusters. Each cluster contains 32 queries and is denoted with a distinct color. Again, we observe that clusters are often local. Indeed, usually consecutive pixels or nearly consecutive pixels are denoted with the same color. For such large number of clusters, it becomes very difficult to extract semantic information from the clustering map without looking at the original image. However, by careful looking at both the attention map and the generated image we can make interesting observations. For instance, we see that distant background pixels are clustered together with much greater frequency compared to other distant non-background pixels. In other words, SMYRF often clusters together background pixels even if they belong to distant grid positions in the generated image (see for example colors in top and last row of the grid).